

Rockchip 智能视频分析（ROCKIVA） SDK开发指南

文件标识：RK-YH-YF-430

发布版本：V2.0.0

日期：2025-09-22

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文提供一个标准模板供套用。后续模板以此份文档为基础改动。

产品版本

芯片名称	内核版本
RV1126B	Linux

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

[illegible]

目录

Rockchip 智能视频分析（ROCKIVA）SDK开发指南

1. 概述
2. 算法模型说明
 - 2.1 检测模型
 - 2.1.1 多合一检测模块
 - 2.1.2 脸人宠
3. SDK使用说明
 - 3.1 ROCKIVA C API SDK开发说明
 - 3.1.1 通用模块
 - 3.1.1.1 功能描述
 - 3.1.1.2 函数接口
 - 3.1.1.2.1 ROCKIVA_Init
 - 3.1.1.2.2 ROCKIVA_Release
 - 3.1.1.2.3 ROCKIVA_SetFrameReleaseCallback
 - 3.1.1.2.4 ROCKIVA_FrameReleaseCallback
 - 3.1.1.2.5 ROCKIVA_PushFrame
 - 3.1.1.2.6 ROCKIVA_PushFrameWithRoi
 - 3.1.1.2.7 ROCKIVA_GetVersion
 - 3.1.1.3 枚举
 - 3.1.1.3.1 RockIvaImageFormat
 - 3.1.1.3.2 RockIvaImageTransform
 - 3.1.1.3.3 RockIvaExecuteStatus
 - 3.1.1.3.4 RockIvaRetCode
 - 3.1.1.3.5 RockIvaLogLevel
 - 3.1.1.3.6 RockIvaObjectType
 - 3.1.1.3.7 RockIvaWorkMode
 - 3.1.1.3.8 RockIvaMemType
 - 3.1.1.4 结构体
 - 3.1.1.4.1 RockIvaPoint
 - 3.1.1.4.2 RockIvaRectangle
 - 3.1.1.4.3 RockIvaQuadrangle
 - 3.1.1.4.4 RockIvaArea
 - 3.1.1.4.5 RockIvaAreas
 - 3.1.1.4.6 RockIvaSize
 - 3.1.1.4.7 RockIvaRectExpandRatio
 - 3.1.1.4.8 RockIvaAngle
 - 3.1.1.4.9 RockIvaImageInfo
 - 3.1.1.4.10 RockIvaImage
 - 3.1.1.4.11 RockIvaObjectInfo
 - 3.1.1.4.12 RockIvaDetectResult
 - 3.1.1.4.13 RockIvaReleaseFrames
 - 3.1.1.4.14 RockIvaInitParam
 - 3.1.2 检测模块
 - 3.1.2.1 功能描述
 - 3.1.2.2 函数接口
 - 3.1.2.2.1 ROCKIVA_DetectResultCallback
 - 3.1.2.2.2 ROCKIVA_DETECT_Init
 - 3.1.2.2.3 ROCKIVA_DETECT_Release
 - 3.1.2.2.4 ROCKIVA_DETECT_Reset
 - 3.1.3 周界模块

- 3.1.3.1 功能描述
- 3.1.3.2 函数接口
 - 3.1.3.2.1 ROCKIVA_BA_ResultCallback
 - 3.1.3.2.2 ROCKIVA_BA_Init
 - 3.1.3.2.3 ROCKIVA_BA_Destroy
 - 3.1.3.2.4 ROCKIVA_BA_Reset
- 3.1.3.3 枚举定义
 - 3.1.3.3.1 RockIvaBaTripEvent
 - 3.1.3.3.2 RockIvaBaRuleObjectFilter
 - 3.1.3.3.3 RockIvaBaRuleTriggerType
- 3.1.3.4 结构体定义
 - 3.1.3.4.1 RockIvaBaWireRule
 - 3.1.3.4.2 RockIvaBaAreaRule
 - 3.1.3.4.3 RockIvaBaTaskRule
 - 3.1.3.4.4 RockIvaBaAiConfig
 - 3.1.3.4.5 RockIvaBaTaskParams
 - 3.1.3.4.6 RockIvaBaTrigger
 - 3.1.3.4.7 RockIvaBaObjectInfo
 - 3.1.3.4.8 RockIvaBaResult
- 3.2 ROCKX2 API调用说明
 - 3.2.1 ROCKIVA算法模块
 - 3.2.1.1 IvaDet
 - 3.2.1.2 IvaTracker
 - 3.2.1.3 IvaPeridet

1. 概述

ROCKIVA SDK是一套智能视频分析（IVA: Intelligence Video Analysis）算法SDK，能够嵌入NVR、IPC等摄像头视频流的产品应用中，提供AI智能算法支持。SDK的主要功能包括目标检测、周界功能、人脸抓拍识别功能、车辆车牌检测识别功能和非机动车检测功能：

- 目标检测功能：检测画面中的人形、人脸、机动车、非机动车和宠物（猫狗）等目标，返回目标位置；
- 周界功能：支持区域入侵、越界检测、区域进入、区域离开等周界功能；

2. 算法模型说明

2.1 检测模型

为了满足多种产品类型的需求，SDK提供了多个目标检测模型，用户可以根据需要选用。
可以直接配置RockIvaInitParam的detModelName为目标检测模型文件名，SDK会自动从配置的模型目录路径下加载对应的模型文件。
也可以通过配置detModel为指定枚举值来加载不同的模型

2.1.1 多合一检测模块

检测类别：人形、人脸、机动车、非机动车、车牌、宠物、人头
detModel配置：ROCKIVA_DET_MODEL_CLS8

模型文件	适合分辨率	内存占用（MB）	Flash占用（MB，未压缩）	耗时（ms）
iva_object_detection_v3_cls8.data	960x540	8.5	4.9	15

2.1.2 脸人宠

检测类别：人形、人脸、宠物
detModel配置：ROCKIVA_DET_MODEL_PFP

模型文件	适合分辨率	内存占用（MB）	Flash占用（MB，未压缩）	耗时（ms）
iva_object_detection_v3_pfp.data	960x540	8.5	4.9	11

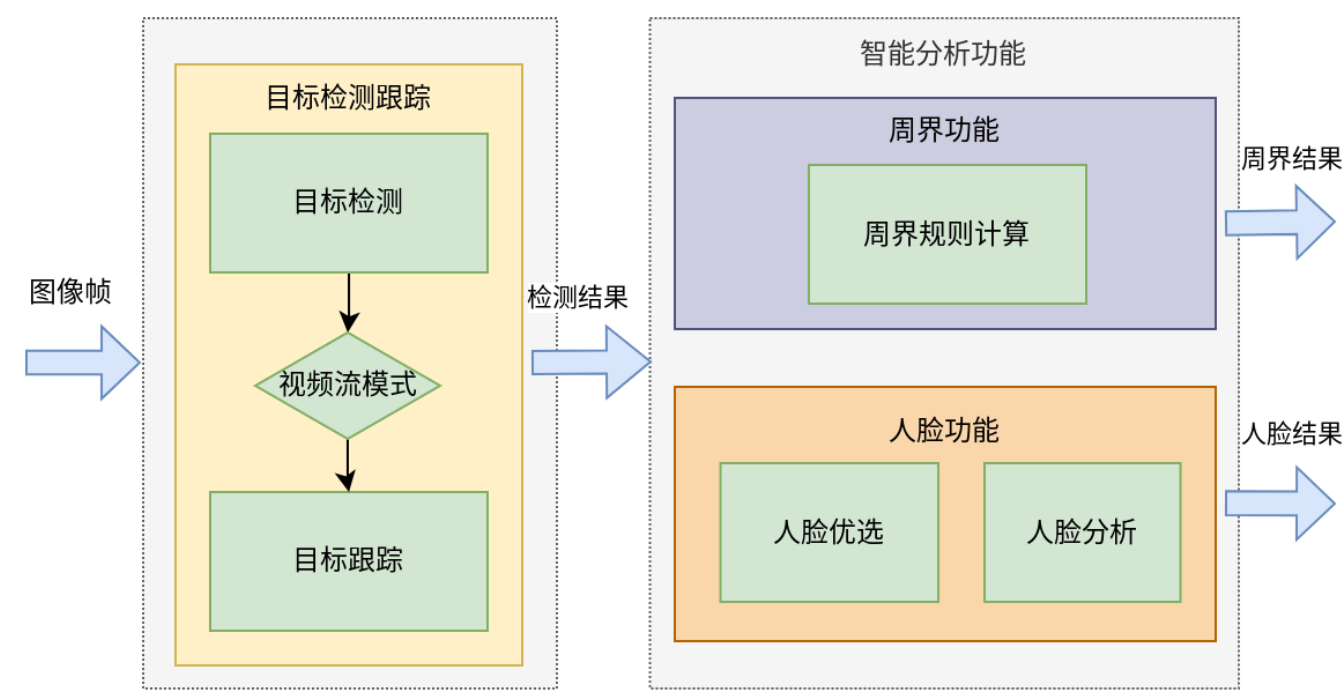
3. SDK使用说明

当前SDK提供两种接口可以使用

- ROCKIVA C API：兼容原ROCKIVA SDK 1.X的接口，原先开发的应用可以直接使用。
- ROCKX2 API：支持更灵活的方式进行调用，能够单独调用算法模块/组合使用。

3.1 ROCKIVA C API SDK开发说明

ROCKIVA C API将视频分析算法流程串起来执行。首先基于目标检测跟踪算法从图像中获取人形、人脸、机动车、非机动车等目标信息，然后根据目标检测跟踪的结果再进行智能分析。当前支持的智能分析功能有周界、人脸、非机动车和车辆车牌，其中当前随系统SDK带的SDK仅支持目标检测和周界算法，其他人脸、非机动车、车辆车牌需要联系FAE获取单独算法SDK。智能分析的每个功能可以独立开启或关闭，也能够同时使用。

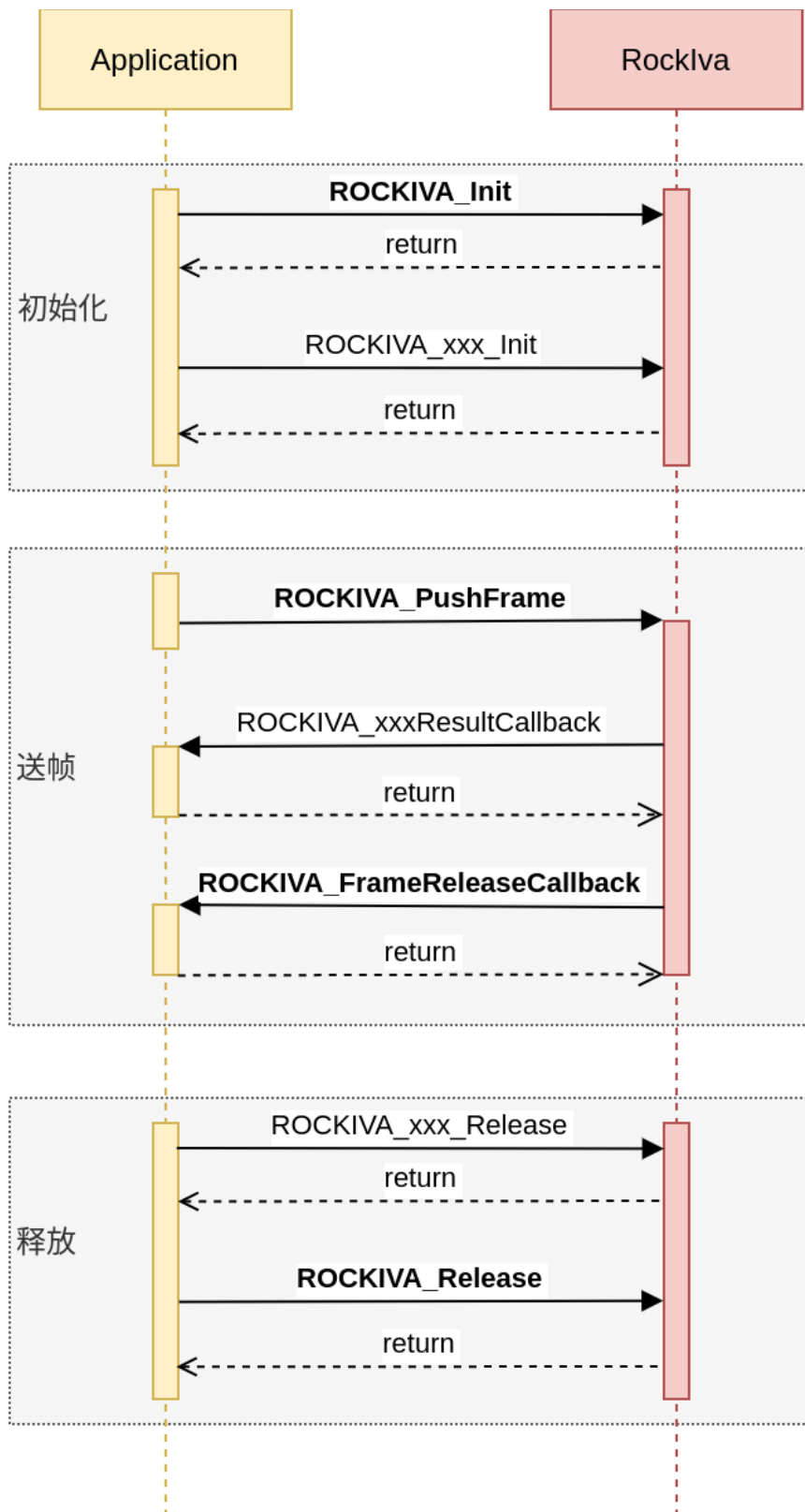


3.1.1 通用模块

3.1.1.1 功能描述

ROCKIVA SDK的接口为异步模式，使用的基本操作就是配置初始化，然后按一定的帧率送图像，在回调函数中获取结果并释放帧内存，最后如果不需要使用就进行释放。

SDK的基本调用流程如图所示。



回调函数ROCKIVA_FrameReleaseCallback用来通知应用可以释放的图像帧，SDK内部会判断如果不需要再使用该图像帧的时候通知外面应用可以进行释放。

对于需要内部缓存帧的功能，比如人脸抓拍，是必须要通过该回调函数进行释放帧；对于检测功能或者周界等内部不需要缓存帧的功能，在获得每帧对应的结果后，SDK内部就不需要再对该图像帧进行操作了，因此也可以直接在结果回调函数中释放。

3.1.1.2 函数接口

3.1.1.2.1 ROCKIVA_Init

【功能】
算法SDK配置初始化，对传入handle进行初始化

【声明】

```
RockIvaRetCode ROCKIVA_Init(RockIvaHandle* handle, RockIvaWorkMode mode, const
RockIvaInitParam* param, void *userdata);
```

【参数】

参数名称	输入/输出	描述
handle	输入	要初始化的实例句柄
mode	输入	工作模式
param	输入	初始化参数
userdata	输入	用户自定义数据

【返回值】
RockIvaRetCode

3.1.1.2.2 ROCKIVA_Release

【功能】
算法SDK销毁释放

【声明】

```
RockIvaRetCode ROCKIVA_Release(RockIvaHandle handle);
```

【参数】

参数名称	输入/输出	描述
handle	输入	要释放的实例句柄

【返回值】
RockIvaRetCode

3.1.1.2.3 ROCKIVA_SetFrameReleaseCallback

【功能】
设置帧释放回调函数

【声明】


```
RockIvaRetCode ROCKIVA_SetFrameReleaseCallback(RockIvaHandle handle,
ROCKIVA_FrameReleaseCallback callback);
```

【参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
callback	输入	回调函数

【返回值】

RockIvaRetCode

3.1.1.2.4 ROCKIVA_FrameReleaseCallback

【功能】

帧释放回调函数

【声明】

```
typedef void (*ROCKIVA_FrameReleaseCallback)(const RockIvaReleaseFrames* releaseFrames, void
*userdata);
```

【参数】

参数名称	输入/输出	描述
RockIvaReleaseFrames	输入	可释放的帧列表
userdata	输入	用户自定义数据

【返回值】

RockIvaRetCode

3.1.1.2.5 ROCKIVA_PushFrame

【功能】

输入图像帧

【声明】

```
RockIvaRetCode ROCKIVA_PushFrame(RockIvaHandle handle, const RockIvaImage* inputImg);
```

【参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
inputImg	输入	输入图像帧

【返回值】

RockIvaRetCode

3.1.1.2.6 ROCKIVA_PushFrameWithRoi

【功能】

输入图像帧（支持设置有效区域）

【声明】

```
RockIvaRetCode ROCKIVA_PushFrameWithRoi(RockIvaHandle handle, const RockIvaImage* inputImg,
const RockIvaAreas* roiAreas);
```

【参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
inputImg	输入	输入图像帧
roiAreas	输入	有效区域

【返回值】

RockIvaRetCode

3.1.1.2.7 ROCKIVA_GetVersion

【功能】

获取SDK版本号

【声明】

```
RockIvaRetCode ROCKIVA_GetVersion(const uint32_t maxLen, char* version);
```

【参数】

参数名称	输入/输出	描述
maxLen	输入	存放版本号buffer大小
version	输入	版本号buffer地址

【返回值】

RockIvaRetCode

3.1.1.3 枚举

枚举	描述
RockIvaImageFormat	图像像素格式
RockIvaImageTransform	输入图像的旋转模式
RockIvaExecuteStatus	执行回调结果状态码
RockIvaRetCode	函数返回错误码
RockIvaLogLevel	日志打印级别
RockIvaObjectType	目标类型
RockIvaWorkMode	工作模式
RockIvaMemType	内存类型

3.1.1.3.1 RockIvaImageFormat

【功能】
图像像素格式类型
【声明】

```
typedef enum {
    ROCKIVA_IMAGE_FORMAT_GRAY8 = 0,
    ROCKIVA_IMAGE_FORMAT_RGB888,
    ROCKIVA_IMAGE_FORMAT_BGR888,
    ROCKIVA_IMAGE_FORMAT_RGBA8888,
    ROCKIVA_IMAGE_FORMAT_BGRA8888,
    ROCKIVA_IMAGE_FORMAT_YUV420P_YU12,
    ROCKIVA_IMAGE_FORMAT_YUV420P_YV12,
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12,
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21,
    ROCKIVA_IMAGE_FORMAT_JPEG,
} RockIvaImageFormat;
```

【成员】

成员	描述
ROCKIVA_IMAGE_FORMAT_GRAY8	单通道灰度图
ROCKIVA_IMAGE_FORMAT_RGB888	三通道彩色图像
ROCKIVA_IMAGE_FORMAT_BGR888	三通道彩色图像
ROCKIVA_IMAGE_FORMAT_RGBA8888	四通道彩色图像
ROCKIVA_IMAGE_FORMAT_BGRA8888	四通道彩色图像
ROCKIVA_IMAGE_FORMAT_YUV420P_YU12	YUV420P_YU12
ROCKIVA_IMAGE_FORMAT_YUV420P_YV12	YUV420P_YV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12	YUV420SP_NV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21	YUV420SP_NV21
ROCKIVA_IMAGE_FORMAT_JPEG	JPEG图像（输入图像不支持该格式）

【注意事项】

无

3.1.1.3.2 RockIvaImageTransform

【功能】

输入图像的旋转模式

【声明】

```
typedef enum {
    ROCKIVA_IMAGE_TRANSFORM_NONE = 0x00,
    ROCKIVA_IMAGE_TRANSFORM_FLIP_H = 0x01,
    ROCKIVA_IMAGE_TRANSFORM_FLIP_V = 0x02,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_90 = 0x04,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_180 = 0x03,
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_270 = 0x07,
} RockIvaImageFormat;
```

【成员】

成员	描述
ROCKIVA_IMAGE_TRANSFORM_NONE	正常
ROCKIVA_IMAGE_TRANSFORM_FLIP_H	水平翻转
ROCKIVA_IMAGE_TRANSFORM_FLIP_V	垂直翻转
ROCKIVA_IMAGE_TRANSFORM_ROTATE_90	顺时针90度
ROCKIVA_IMAGE_TRANSFORM_ROTATE_180	顺时针180度
ROCKIVA_IMAGE_TRANSFORM_ROTATE_270	顺时针270度

3.1.1.3.3 RockIvaExecuteStatus

- 【功能】
执行回调结果状态码
- 【声明】

```
typedef enum {
    ROCKIVA_SUCCESS = 0,
    ROCKIVA_UNKNOWN = 1,
    ROCKIVA_NULL_PTR = 2,
    ROCKIVA_ALLOC_FAILED = 3,
    ROCKIVA_INVALID_INPUT = 4,
    ROCKIVA_EXECUTE_FAILED = 6,
    ROCKIVA_NOT_CONFIGURED = 7,
    ROCKIVA_NO_CAPACITY = 8,
    ROCKIVA_BUFFER_FULL = 9,
    ROCKIVA_LICENSE_ERROR = 10,
    ROCKIVA_JPEG_DECODE_ERROR = 11,
    ROCKIVA_DECODER_EXIT = 12,
} RockIvaExecuteStatus;
```

- 【成员】

成员	描述
ROCKIVA_SUCCESS	运行结果正常
ROCKIVA_UNKNOWN	错误类型未知
ROCKIVA_NULL_PTR	操作空指针
ROCKIVA_ALLOC_FAILED	申请空间失败
ROCKIVA_INVALID_INPUT	无效的输入
ROCKIVA_EXECUTE_FAILED	内部执行错误
ROCKIVA_NOT_CONFIGURED	未配置的类型
ROCKIVA_NO_CAPACITY	业务已建满
ROCKIVA_BUFFER_FULL	缓存区域满
ROCKIVA_LICENSE_ERROR	license异常
ROCKIVA_JPEG_DECODE_ERROR	解码出错
ROCKIVA_DECODER_EXIT	解码器内部退出

3.1.1.3.4 RockIvaRetCode

【功能】

函数调用返回错误码

【声明】

```
typedef enum {
    ROCKIVA_RET_SUCCESS = 0,
    ROCKIVA_RET_FAIL = -1,
    ROCKIVA_RET_NULL_PTR = -2,
    ROCKIVA_RET_INVALID_HANDLE = -3,
    ROCKIVA_RET_LICENSE_ERROR = -4,
    ROCKIVA_RET_UNSUPPORTED = -5,
    ROCKIVA_RET_STREAM_SWITCH = -6,
    ROCKIVA_RET_BUFFER_FULL = -7,
} RockIvaRetCode;
```

【成员】

成员	描述
ROCKIVA_RET_SUCCESS	成功
ROCKIVA_RET_FAIL	失败
ROCKIVA_RET_NULL_PTR	空指针
ROCKIVA_RET_INVALID_HANDLE	无效句柄
ROCKIVA_RET_LICENSE_ERROR	License错误
ROCKIVA_RET_UNSUPPORTED	不支持
ROCKIVA_RET_STREAM_SWITCH	码流切换
ROCKIVA_RET_BUFFER_FULL	缓存区域满

3.1.1.3.5 RockIvaLogLevel

- 【功能】
日志打印级别
- 【声明】

```
typedef enum {  
    ROCKIVA_LOG_ERROR = 0,  
    ROCKIVA_LOG_WARN = 1,  
    ROCKIVA_LOG_DEBUG = 2,  
    ROCKIVA_LOG_INFO = 3,  
    ROCKIVA_LOG_TRACE = 4  
} RockIvaLogLevel;
```

- 【成员】

成员	描述
ROCKIVA_LOG_ERROR	错误级别（默认）
ROCKIVA_LOG_WARN	警告级别
ROCKIVA_LOG_DEBUG	调试级别
ROCKIVA_LOG_INFO	信息级别
ROCKIVA_LOG_TRACE	跟踪调用级别

- 【说明】
日志等级数值越大，打印的日志越多。

3.1.1.3.6 RockIvaObjectType

【功能】
检测的目标类别
【声明】

```
typedef enum {  
    ROCKIVA_OBJECT_TYPE_NONE = 0x0,  
    ROCKIVA_OBJECT_TYPE_PERSON = 0x1,  
    ROCKIVA_OBJECT_TYPE_VEHICLE = 0x2,  
    ROCKIVA_OBJECT_TYPE_NON_VEHICLE = 0x4,  
    ROCKIVA_OBJECT_TYPE_FACE = 0x8,  
    ROCKIVA_OBJECT_TYPE_HEAD = 0x10,  
    ROCKIVA_OBJECT_TYPE_PET = 0x20,  
    ROCKIVA_OBJECT_TYPE_MOTORCYCLE = 0x40,  
    ROCKIVA_OBJECT_TYPE_BICYCLE = 0x80,  
    ROCKIVA_OBJECT_TYPE_PLATE = 0x100  
} RockIvaObjectType;
```

【成员】

成员	描述
ROCKIVA_OBJECT_TYPE_NONE	未知
ROCKIVA_OBJECT_TYPE_PERSON	人形
ROCKIVA_OBJECT_TYPE_VEHICLE	机动车
ROCKIVA_OBJECT_TYPE_NON_VEHICLE	非机动车
ROCKIVA_OBJECT_TYPE_FACE	人脸
ROCKIVA_OBJECT_TYPE_HEAD	人头
ROCKIVA_OBJECT_TYPE_PET	猫、狗
ROCKIVA_OBJECT_TYPE_MOTORCYCLE	电瓶车
ROCKIVA_OBJECT_TYPE_BICYCLE	自行车
ROCKIVA_OBJECT_TYPE_PLATE	车牌

3.1.1.3.7 RockIvaWorkMode

【功能】
工作模式
【声明】

```
typedef enum {  
    ROCKIVA_MODE_VIDEO    = 0,  
    ROCKIVA_MODE_PICTURE = 1,  
} RockIvaWorkMode;
```


【成员】

成员	描述
ROCKIVA_MODE_VIDEO	视频流模式(使能跟踪)
ROCKIVA_MODE_PICTURE	图片模式(不使能跟踪)

【说明】

周界功能必须要工作在视频流模式；检测和人脸功能可以使用视频流或图片模式；

3.1.1.3.8 RockIvaMemType

【功能】

内存类型

【声明】

```
typedef enum {
    ROCKIVA_MEM_TYPE_CPU = 0,
    ROCKIVA_MEM_TYPE_DMA = 1
} RockIvaMemType;
```

【成员】

成员	描述
ROCKIVA_MEM_TYPE_CPU	虚拟地址内存
ROCKIVA_MEM_TYPE_DMA	DMA BUF内存（RV1106）

【说明】

开发Demo时预开辟的输入图像内存空间类型，对于RV1106平台RGA需要使用DMA BUF内存类型。

3.1.1.4 结构体

3.1.1.4.1 RockIvaPoint

【功能】

点坐标

【声明】

```
typedef struct {
    uint16_t x;
    uint16_t y;
} RockIvaPoint;
```

【成员】

成员	描述
x	横坐标，万分比表示，0~9999有效
y	纵坐标，万分比表示，0~9999有效
【功能】	
线坐标	
【声明】	

```
typedef struct {
    RockIvaPoint head;
    RockIvaPoint tail;
} RockIvaLine;
```

【成员】

成员	描述
head	头坐标
tail	尾坐标

3.1.1.4.2 RockIvaRectangle

【功能】

正四边形坐标

【声明】

```
typedef struct {
    RockIvaPoint topLeft;
    RockIvaPoint bottomRight;
} RockIvaRectangle;
```

【成员】

成员	描述
topLeft	左上角坐标
bottomRight	右下角坐标

3.1.1.4.3 RockIvaQuadrangle

【功能】

任意四边形坐标

【声明】

```
typedef struct {
    RockIvaPoint topLeft;
    RockIvaPoint topRight;
    RockIvaPoint bottomLeft;
    RockIvaPoint bottomRight;
} RockIvaQuadrangle;
```

【成员】

成员	描述
topLeft	左上角坐标
topRight	右上角坐标
bottomLeft	左下角坐标
bottomRight	右下角坐标

3.1.1.4.4 RockIvaArea

【功能】

多边形区域坐标

【声明】

```
typedef struct {
    uint32_t pointNum;
    RockIvaPoint points[ROCKIVA_AREA_POINT_NUM_MAX];
} RockIvaArea;
```

【成员】

成员	描述
pointNum	点个数
points	围成区域的所有点坐标

3.1.1.4.5 RockIvaAreas

【功能】

多区域

【声明】

```
typedef struct {
    uint32_t areaNum;
    RockIvaArea areas[ROCKIVA_AREA_NUM_MAX];
} RockIvaAreas;
```

【成员】

成员	描述
areaNum	区域数量
areas	区域数组

3.1.1.4.6 RockIvaSize

【功能】
目标大小向四周
【声明】

```
typedef struct {
    uint16_t width;
    uint16_t height;
} RockIvaSize;
```

【成员】

成员	描述
width	宽度（万分比坐标，值范围0~9999）
height	高度（万分比坐标，值范围0~9999）

3.1.1.4.7 RockIvaRectExpandRatio

【功能】
检测框向四周扩展的比例大小配置
【声明】

```
typedef struct {
    float up;
    float down;
    float left;
    float right;
} RockIvaRectExpandRatio;
```

【成员】

成员	描述
up	检测框向上按框高扩展的比例大小
down	检测框向下按框高扩展的比例大小
left	检测框向左按框宽扩展的比例大小
right	检测框向右按框宽扩展的比例大小

3.1.1.4.8 RockIvaAngle

【功能】
人脸角度信息
【声明】

```
typedef struct {  
    int16_t pitch;  
    int16_t yaw;  
    int16_t roll;  
} RockIvaAngle;
```

【成员】

成员	描述
pitch	俯仰角,表示绕x轴旋转角度
yaw	偏航角,表示绕y轴旋转角度
roll	翻滚角,表示绕z轴旋转角度

3.1.1.4.9 RockIvaImageInfo

【功能】
图像信息
【声明】

```
typedef struct {  
    uint16_t width;  
    uint16_t height;  
    RockIvaImageFormat format;  
    RockIvaImageTransform transformMode;  
} RockIvaImageInfo;
```

【成员】

成员	描述
width	图像宽度
height	图像高度
format	图像像素格式
transformMode	旋转模式

3.1.1.4.10 RockIvaImage

【功能】

图像

【声明】

```
typedef struct {
    uint32_t frameId;
    uint32_t channelId;
    RockIvaImageInfo info;
    uint32_t size;
    uint8_t* dataAddr;
    uint8_t* dataPhyAddr;
    int32_t dataFd;
    void* extData;
} RockIvaImage;
```

【成员】

成员	描述
frameId	原始采集帧序号（应用自定义）
channelId	通道号（应用自定义，单通道置为0）
info	图像信息
size	图像数据大小，图像格式为JPEG时需要
dataAddr	输入图像数据虚地址
dataPhyAddr	输入图像数据物理地址（没有用置为NULL）
dataFd	输入图像数据fd（没有用置为0）
extData	用户自定义扩展数据

3.1.1.4.11 RockIvaObjectInfo

【功能】

单个目标检测结果信息

【声明】

```
typedef struct {
    uint32_t objId;
    uint32_t frameId;
    uint32_t score;
    RockIvaRectangle rect;
    RockIvaObjectType type;
} RockIvaObjectInfo;
```

【成员】

成员	描述
objId	目标ID[0~232)
frameId	帧ID
score	目标检测分数 [1-100]
rect	目标区域框 (万分比)
type	目标类别

3.1.1.4.12 RockIvaDetectResult

【功能】
目标检测结果
【声明】

```
typedef struct {
    uint32_t frameId;
    RockIvaImage frame;
    uint32_t channelId;
    uint32_t objNum;
    RockIvaObjectInfo objInfo[ROCKIVA_MAX_OBJ_NUM];
} RockIvaDetectResult;
```

【成员】

成员	描述
frameId	帧ID
frame	对应的输入图像帧
channelId	通道ID
objNum	目标个数
objInfo	各目标检测信息

3.1.1.4.13 RockIvaReleaseFrames

【功能】
可以释放的帧列表
【声明】

```
typedef struct {
    uint32_t channelId;
    uint32_t count;
    RockIvaImage frameId[ROCKIVA_MAX_OBJ_NUM];
} RockIvaReleaseFrames;
```

【成员】

成员	描述
channelId	通道ID
count	数量
frameId	可释放的帧

3.1.1.4.14 RockIvaInitParam

【功能】
算法全局参数配置
【声明】

```
typedef struct {
    RockIvaLogLevel RockIvaLogLevel;
    char logPath[ROCKIVA_PATH_LENGTH];
    char modelPath[ROCKIVA_PATH_LENGTH];
    RockIvaMemInfo license;
    uint32_t coreMask;
    uint32_t channelId;
    RockIvaImageInfo imageInfo;
    RockIvaAreas roiAreas;
    uint32_t detObjectType;
} RockIvaInitParam;
```

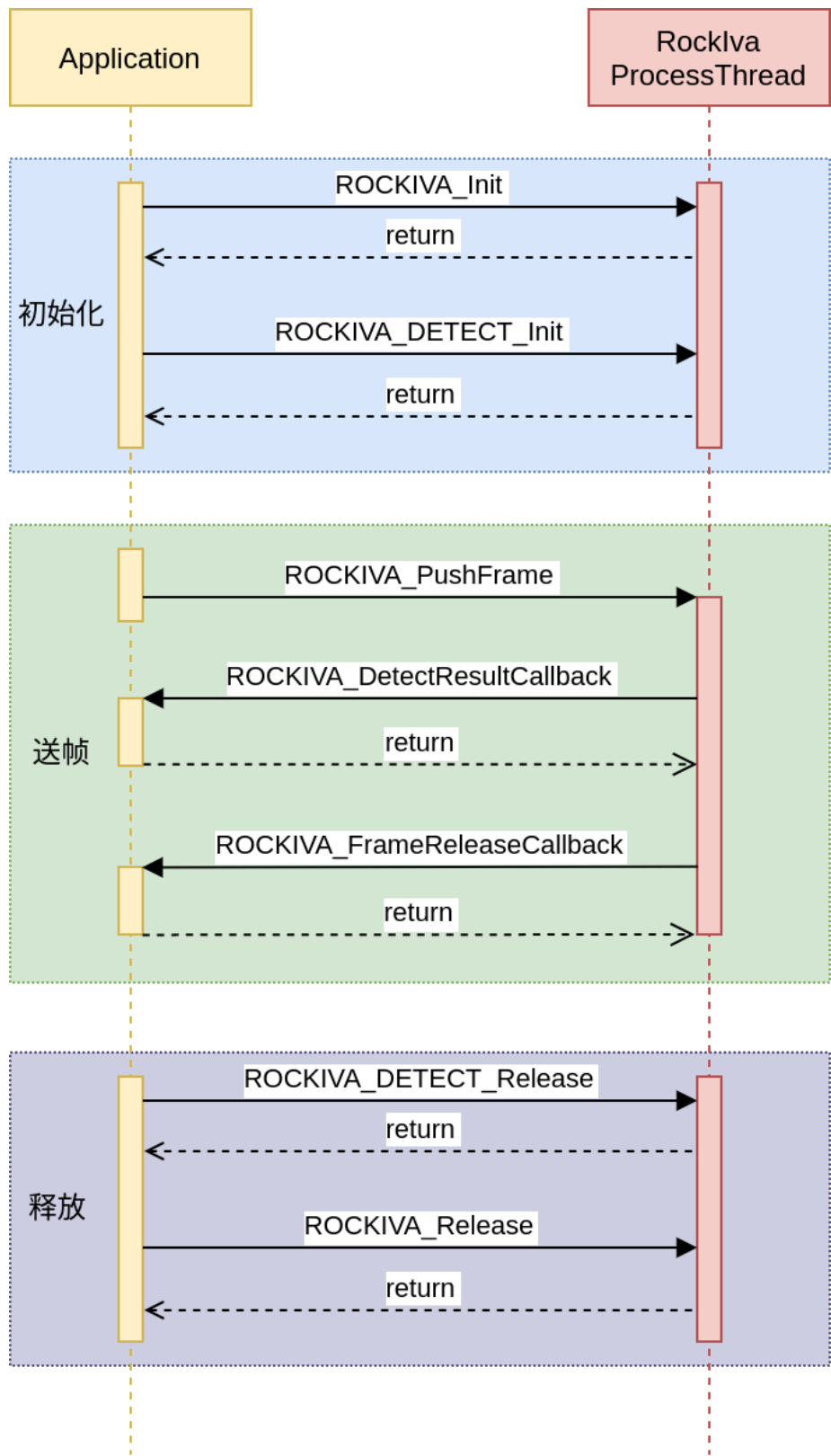
【成员】

成员	描述
RockIvaLogLevel	日志等级
logPath	日志输出路径
modelPath	存放算法模型路径
license	License信息
coreMask	指定使用哪个NPU核跑(仅RK3588平台有效)
channelId	通道号
imageInfo	输入图像信息
roiAreas	有效区域
detObjectType	配置要检测的目标： 例如检测人\机动车\非机动车： ROCKIVA_OBJECT_TYPE_PERSON ROCKIVA_OBJECT_TYPE_VEHICLE ROCKIVA_OBJECT_TYPE_NON_VEHICLE

3.1.2 检测模块

3.1.2.1 功能描述

检测功能可以通过RockIvaInitParam参数配置要检测的目标类型包括人脸、人形、机动车、非机动车和宠物猫狗，设置检测结果回调。基本调用流程如图所示。



配置初始化后，SDK内部会创建线程异步处理每一帧，应用每次推送的图像帧都会放到一个缓存队列中送给处理线程。需要注意的是结果回调函数ROCKIVA_DetectResultCallback是运行在该内部处理线程，因此尽量不要做耗时太久的操作，否则会影响处理帧率。

检测模块可以工作在视频流模式或图片模式。视频流模式下会使能目标跟踪，检测回调中的每个检测目标的objId有效；图片模式下会关闭目标跟踪，检测目标的objId值无效。

对于用户推送进来的每一帧需要通过ROCKIVA_FrameReleaseCallback回调函数返回的结果进行释放。

3.1.2.2 函数接口

接口	描述
ROCKIVA_DetectResultCallback	结果回调函数
ROCKIVA_DETECT_Init	初始化
ROCKIVA_DETECT_Release	释放
ROCKIVA_DETECT_Reset	重置

3.1.2.2.1 ROCKIVA_DetectResultCallback

【功能】
结果回调函数
【声明】

```
typedef void (*ROCKIVA_DetectResultCallback)(  
    const RockIvaDetectResult* result,  
    const RockIvaExecuteStatus status,  
    void* userdata);
```

【输入参数】

参数名称	输入/输出	描述
result	输入	结果
status	输入	状态码
userdata	输入	用户自定义数据

【返回值】
RockIvaRetCode

3.1.2.2.2 ROCKIVA_DETECT_Init

【功能】
初始化
【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Init(RockIvaHandle handle, const
ROCKIVA_DetectResultCallback resultCallback);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
resultCallback	输入	结果回调函数

【返回值】

RockIvaRetCode

3.1.2.2.3 ROCKIVA_DETECT_Release

【功能】

释放

【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Release(RockIvaHandle handle);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	实例句柄

【返回值】

RockIvaRetCode

3.1.2.2.4 ROCKIVA_DETECT_Reset

【功能】

重置

【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Reset(RockIvaHandle handle);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要重置的handle，运行时重新配置(重新配置会导致内部的一些记录清空复位，但是模型不会重新初始化)

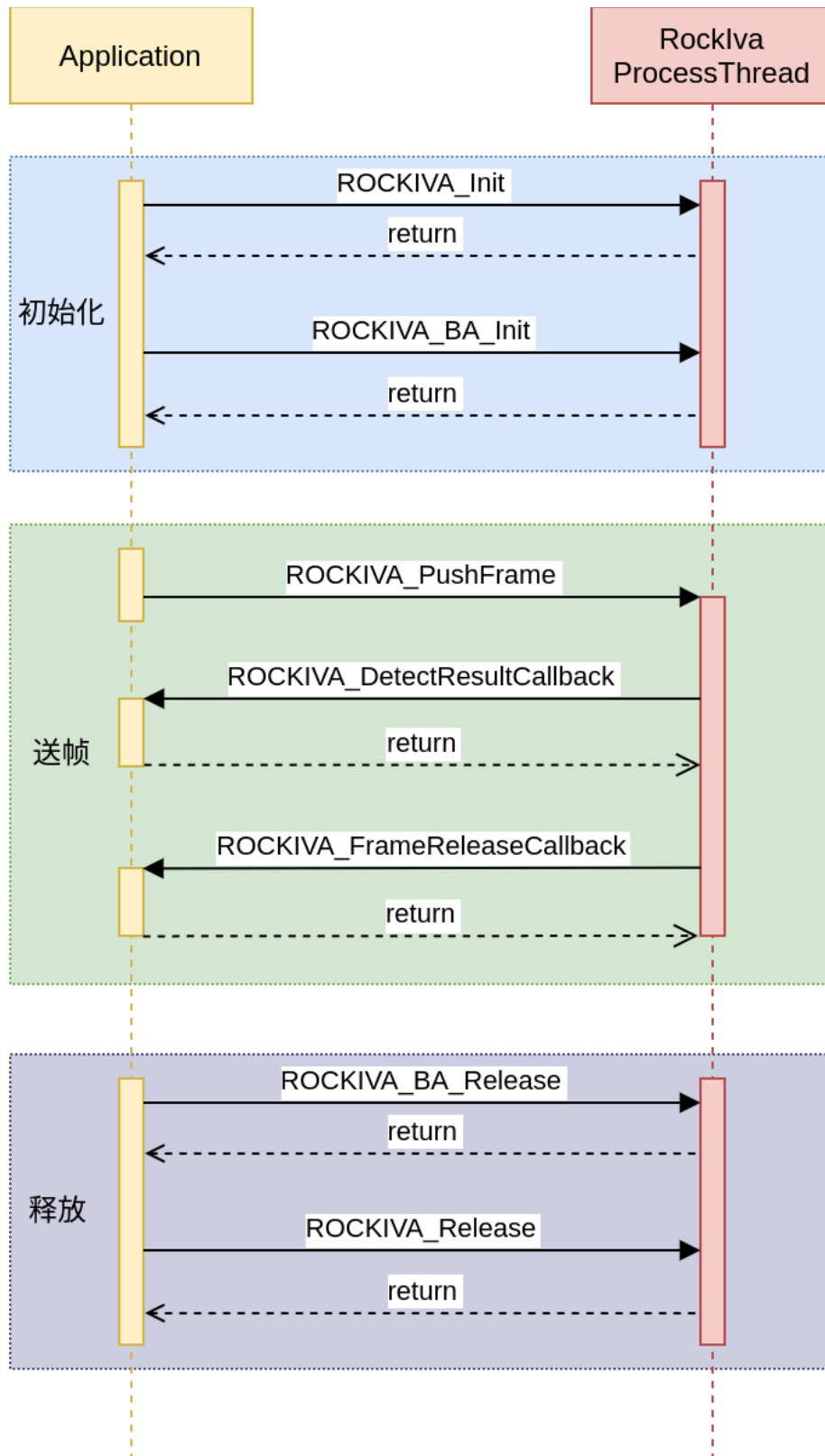
【返回值】

RockIvaRetCode

3.1.3 周界模块

3.1.3.1 功能描述

周界功能的流程基本和检测类似，通过ROCKIVA_BA_Init初始化周界相关配置以及配置周界结果回调。
周界功能模块代码调用流程如图所示。



注意周界的工作模式必须要是视频流模式（`ROCKIVA_MODE_VIDEO`），因为周界的规则判断需要根据目标前后帧的坐标变化进行计算。

周界功能有四种类型：

- 区域入侵：目标进入设定区域并停留超过设定时间触发
- 区域进入：目标从设定区域外进入区域内触发
- 区域离开：目标从设定区域内离开区域外触发
- 越界检测：目标从设定的线段越过（符合设定方向）触发

可配置规则主要有：

规则配置	说明
区域	设定规则区域：有顺序的最多6个点构成的多边形
界线	设定规则界线：两个点构成的线段，可设置方向
目标类型过滤	设定触发的目标类型：人形、车辆、非机动车
目标大小过滤	设定目标大小：最小和最大宽高

区域入侵结果展示如图所示。目标进入禁区后会上报事件。

区域入侵进入禁区前

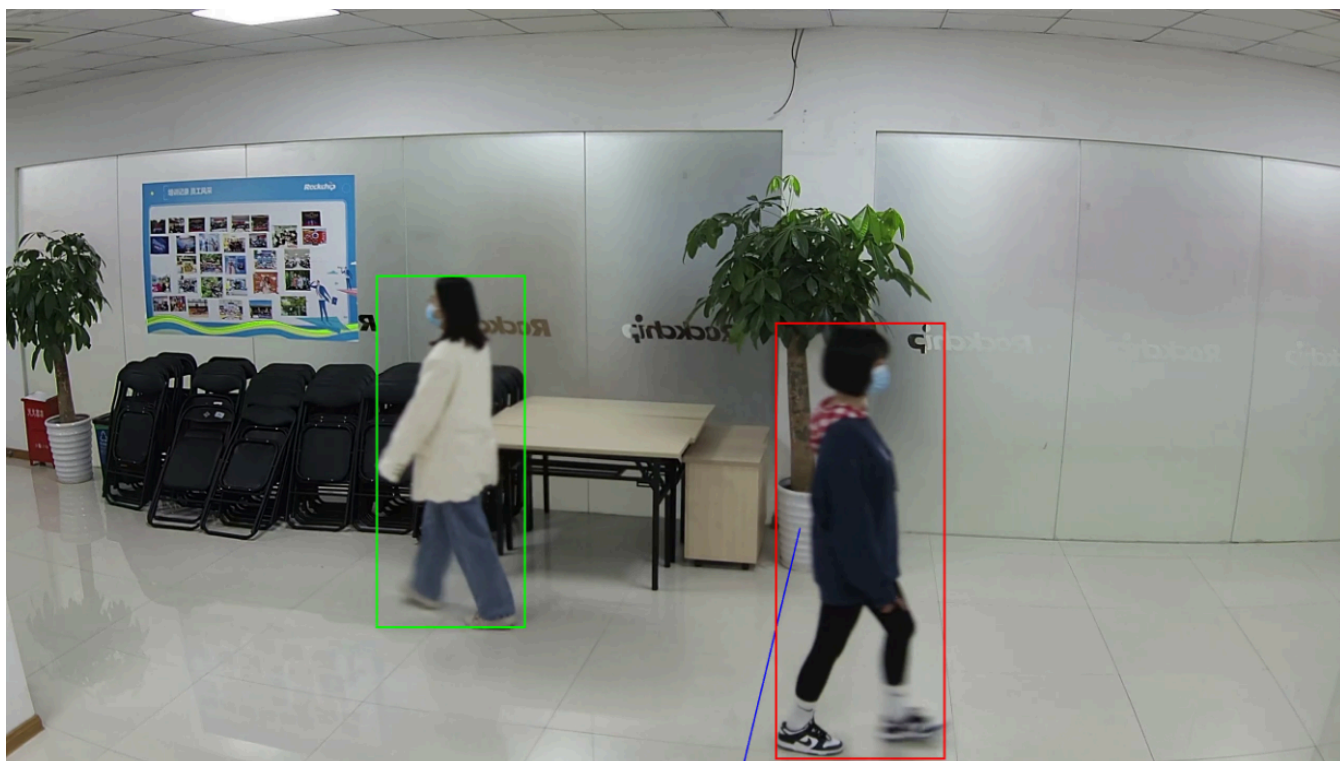


区域入侵进入禁区后



越界结果展示如图所示。目标由禁止的方向越过边界线时会上报事件。

越界结果



3.1.3.2 函数接口

接口	描述
ROCKIVA_BA_ResultCallback	结果回调函数
ROCKIVA_BA_Init	初始化
ROCKIVA_BA_Destroy	销毁
ROCKIVA_BA_Reset	重置

3.1.3.2.1 ROCKIVA_BA_ResultCallback

【功能】
结果回调函数
【声明】

```
typedef void(*ROCKIVA_BA_ResultCallback)(  
    const RockIvaBaResult *result,  
    const RockIvaExecuteStatus status,  
    void *userdata);
```

【输入参数】

参数名称	输入/输出	描述
result	输入	结果
status	输入	状态码
userdata	输入	用户自定义数据

【返回值】
RockIvaRetCode

3.1.3.2.2 ROCKIVA_BA_Init

【功能】
初始化
【声明】

```
RockIvaRetCode ROCKIVA_BA_Init(RockIvaHandle handle,  
                                const RockIvaBaTaskParams *initParams,  
                                const ROCKIVA_BA_ResultCallback resultCallback);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要初始化的handle
initParams	输入	初始化参数配置
resultCallback	输入	回调函数

【返回值】

RockIvaRetCode

3.1.3.2.3 ROCKIVA_BA_Destroy

【功能】

销毁

【声明】

```
RockIvaRetCode ROCKIVA_BA_Destroy(RockIvaHandle handle);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要销毁的handle
【返回值】		
RockIvaRetCode		

3.1.3.2.4 ROCKIVA_BA_Reset

【功能】

重置

【声明】

```
RockIvaRetCode ROCKIVA_BA_Reset(RockIvaHandle handle, const RockIvaBaTaskParams* params);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要重置的handle
params	输入	新的初始化参数配置

【返回值】

RockIvaRetCode

3.1.3.3 枚举定义

枚举	描述
RockIvaBaTripEvent	周界规则类型（绊线/区域事件）
RockIvaBaRuleObjectFilter	触发规则目标类型过滤
RockIvaBaRuleTriggerType	目标规则触发类型

3.1.3.3.1 RockIvaBaTripEvent

【功能】
周界规则类型（绊线/区域事件）
【声明】

```
typedef enum {
    ROCKIVA_BA_TRIP_EVENT_BOTH = 0,
    ROCKIVA_BA_TRIP_EVENT_DEASIL = 1,
    ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES = 2,
    ROCKIVA_BA_TRIP_EVENT_IN = 3,
    ROCKIVA_BA_TRIP_EVENT_OUT = 4,
    ROCKIVA_BA_TRIP_EVENT_STAY = 5,
} RockIvaBaTripEvent;
```

【成员】

成员	描述
ROCKIVA_BA_TRIP_EVENT_BOTH	绊线:双向触发
ROCKIVA_BA_TRIP_EVENT_DEASIL	绊线:顺时针触发
ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES	绊线:逆时针触发
ROCKIVA_BA_TRIP_EVENT_IN	进入区域
ROCKIVA_BA_TRIP_EVENT_STAY	区域入侵

3.1.3.3.2 RockIvaBaRuleObjectFilter

【功能】
触发规则目标类型过滤
【声明】

```
typedef enum {
    ROCKIVA_BA_RULE_OBJ_NONE = 0,
    ROCKIVA_BA_RULE_OBJ_VEHICLE = 1,
    ROCKIVA_BA_RULE_OBJ_NONVEHICLE = 2,
    ROCKIVA_BA_RULE_OBJ_PERSON = 4,
    ROCKIVA_BA_RULE_OBJ_FULL = 7,
} RockIvaBaRuleObjectFilter;
```

【成员】

成员	描述
ROCKIVA_BA_RULE_OBJ_NONE	没有目标
ROCKIVA_BA_RULE_OBJ_VEHICLE	使能机动车目标
ROCKIVA_BA_RULE_OBJ_NONVEHICLE	使能非机动车目标
ROCKIVA_BA_RULE_OBJ_PERSON	使能人形目标
ROCKIVA_BA_RULE_OBJ_FULL	全部使能

3.1.3.3.3 RockIvaBaRuleTriggerType

【功能】

目标规则触发类型

【声明】

```
typedef enum {
    ROCKIVA_BA_RULE_NONE = 0b000000000000,
    ROCKIVA_BA_RULE_CROSS = 0b000000000001,
    ROCKIVA_BA_RULE_INAREA = 0b000000000010,
    ROCKIVA_BA_RULE_OUTAREA = 0b000000000100,
    ROCKIVA_BA_RULE_STAY = 0b000000001000,
} RockIvaBaRuleTriggerType;
```

【成员】

成员	描述
ROCKIVA_BA_RULE_NONE	
ROCKIVA_BA_RULE_CROSS	拌线
ROCKIVA_BA_RULE_INAREA	进入区域
ROCKIVA_BA_RULE_OUTAREA	离开区
ROCKIVA_BA_RULE_STAY	区域入侵

3.1.3.4 结构体定义

结构体	描述
RockIvaBaWireRule	越界规则
RockIvaBaAreaRule	区域规则
RockIvaBaTaskRule	行为分析类规则配置
RockIvaBaAiConfig	算法配置
RockIvaBaTaskParams	行为分析业务初始化参数配
RockIvaBaTrigger	第一次触发规则信息
RockIvaBaObjectInfo	单个目标检测基本信息
RockIvaBaResult	检测结果全部信息

3.1.3.4.1 RockIvaBaWireRule

【功能】
越界规则
【声明】

```
typedef struct {
    uint8_t ruleEnable;
    uint32_t ruleID;
    RockIvaLine line;
    RockIvaLine directLine;
    RockIvaBaTripEvent event;
    RockIvaSize minObjSize[3];
    RockIvaSize maxObjSize[3];
    uint32_t objType;
    uint8_t rulePriority;
    uint8_t sense;
    uint8_t triggerMode;
} RockIvaBaWireRule;
```

【成员】

成员	描述
ruleEnable	规则是否启用，1->启用，0->不启用
ruleID	规则ID，有效范围[0, 3]
line	越界线配置
directLine	方向线配置
event	越界方向
minObjSize	万分比表示 最小目标: 0机动车 1非机动车 2行人
maxObjSize	万分比表示 最大目标: 0机动车 1非机动车 2行人
objType	置触发目标类型： RockIvaBaRuleObjectFilter 例：车、人： RULE_OBJ_VEHICLE RULE_OBJ_PERSON
rulePriority	规则优先级： 0 高， 1 中， 2 低
sense	灵敏度,1~100
trigerMode	触发模式： 0： 目标仅触发一次； 1： 目标每次越界都触发

3.1.3.4.2 RockIvaBaAreaRule

- 【功能】
- 区域规则
- 【声明】

```
typedef struct {
    uint8_t ruleEnable;
    uint32_t ruleID;
    RockIvaArea area;
    RockIvaBaTripEvent event;
    RockIvaSize minObjSize[3];
    RockIvaSize maxObjSize[3];
    uint32_t objType;
    uint32_t alertTime;
    uint8_t sense;
    uint8_t checkEnter;
} RockIvaBaAreaRule;
```

- 【成员】

成员	描述
ruleEnable	规则是否启用，1->启用，0->不启用
ruleID	规则ID，有效范围[1, 256]
area	区域配置
event	区域事件
minObjSize	万分比表示 最小目标: 0机动车 1非机动车 2行人
maxObjSize	万分比表示 最大目标: 0机动车 1非机动车 2行人
objType	配置触发目标类型： RockIvaBaRuleObjectFilter 例：车、人： RULE_OBJ_VEHICLE RULE_OBJ_PERSON
alertTime	告警时间设置
sense	灵敏度,1~100
checkEnter	区域入侵规则是否需要检查目标有进入 [0: 不启用， 1： 启用]

3.1.3.4.3 RockIvaBaTaskRule

- 【功能】
周界规则配置
- 【声明】

```
typedef struct {  
    RockIvaBaWireRule tripWireRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaInRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaOutRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaInBreakRule[ROCKIVA_BA_MAX_RULE_NUM];  
} RockIvaBaTaskRule;
```

- 【成员】

成员	描述
tripWireRule	越界事件
areaInRule	进入区域
areaOutRule	离开区域
areaInBreakRule	区域入侵

3.1.3.4.4 RockIvaBaAiConfig

- 【功能】
算法配置
- 【声明】

```
typedef struct {
    uint8_t filterPersonMode;
    uint8_t detectResultMode;
} RockIvaBaAiConfig;
```

【成员】

成员	描述
filterPersonMode	过滤非机动车/机动车驾驶员人形结果 0: 不过滤; 1: 过滤
detectResultMode	上报目标检测结果模式 0: 不上报没有触发规则的检测目标; 1: 上报没有触发规则的检测目标

3.1.3.4.5 RockIvaBaTaskParams

【功能】

周界初始化参数配置

【声明】

```
typedef struct {
    RockIvaBaTaskRule baRules;
    RockIvaBaAiConfig aiConfig;
} RockIvaBaTaskParams;
```

【成员】

成员	描述
baRules	周界规则参数配置初始化
aiConfig	算法配置

3.1.3.4.6 RockIvaBaTrigger

【功能】

目标第一次触发规则信息，可以用于抓拍

【声明】

```
typedef struct {
    int32_t ruleID;
    RockIvaBaRuleTriggerType triggerType;
} RockIvaBaTrigger;
```

【成员】

成员	描述
ruleID	触发规则ID
triggerType	触发规则类型

3.1.3.4.7 RockIvaBaObjectInfo

【功能】
触发周界规则的单个目标信息
【声明】

```
typedef struct {
    RockIvaObjectInfo objInfo;
    uint32_t triggerRules;
    RockIvaBaTrigger firstTrigger;
} RockIvaBaObjectInfo;
```

【成员】

成员	描述
objInfo	目标检测结果信息
triggerRules	目标触发的规则
firstTrigger	目标第一次触发的规则

3.1.3.4.8 RockIvaBaResult

【功能】
检测结果全部信息
【声明】

```
typedef struct {
    uint32_t frameId;
    RockIvaImage frame;
    uint32_t channelId;
    uint32_t objNum;
    RockIvaObjectInfo triggerObjects [ROCKIVA_MAX_OBJ_NUM];
} RockIvaBaResult;
```

【成员】

成员	描述
frameId	输入图像帧ID
frame	对应的输入图像帧
channelId	通道号
objNum	目标个数
triggerObjects	触发周界规则的目标

3.2 ROCKX2 API调用说明

ROCKX2 API是新推出的通用的算法SDK接口框架，客户可以通过一套统一的接口调用对算法SDK进行加载、调用。详细SDK的开发说明，请参考《Rockchip_Developer_Guide_ROCKX2_SDK_CN》

3.2.1 ROCKIVA算法模块

算法模块	说明
IvaDet	目标检测算法模块
IvaTracker	目标跟踪算法模块
IvaPeridet	周界算法模块

3.2.1.1 IvaDet

【功能】

目标检测

【模块名】

IvaDet

【参数】

参数名称	类型	说明
model_name	char*	模型名称（模型需要放置于配置的数据_path目录下）
model_path	char*	模型绝对路径

【输入】

参数名称	类型	说明
image	RockXImage	输入图像
roi	RockXRect	输入图像ROI（可以为NULL）

【输出】

参数输出	类型	说明
detect_result	RockIvaDetectResult	检测结果

【示例】

--

```

int RockXIvaInitTaskDet(RockXHandle* rockxhandle, const char* det_model_name)
{
    RockXRetCode ret;
    // create handle
    RockXHandle handle = RockXCreate();

    // set config
    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "IvaDet", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_DATA_PATH, "/data/rockiva_data", 0);

    RockXSetParamString(handle, NULL, ROCKX_KEY_AUTH_CONFIG, "/data/auth_config.json", 0);

    // set rknn model name
    ret = RockXSetParamString(handle, "IvaDet", "model_name", det_model_name, 0);
    // init
    ret = RockXInit(handle);
    if (ret != ROCKX_RET_SUCCESS) {
        printf("RockXInit fail, ret=%d\n", ret);
        return ret;
    }

    *rockxhandle = handle;

    return 0;
}

int RockXIvaReleaseTaskDet(RockXHandle* rockxhandle)
{
    RockXDestroy(*rockxhandle);
    *rockxhandle = NULL;
    return 0;
}

int main(int argc, char* argv[]) {
    char* det_model_name = argv[1];
    char* image_path = argv[2];

    int ret;

    // create handle
    RockXHandle handle;
    ret = RockXIvaInitTaskDet(&handle, det_model_name);
    if (ret < 0) {
        printf("RockXIvaInitTaskDet fail, ret=%d\n", ret);
        return ret;
    }

    // read image
    RockXImage image;
    memset(&image, 0, sizeof(RockXImage));
    read_image_jpeg(image_path, &image);

```

```

RockIvaDetectResult det_result;
ret = RockXIvaCallTaskDet(handle, &image, &det_result);
if (ret < 0) {
    printf("RockXIvaCallTaskDet fail, ret=%d\n", ret);
    return ret;
}

for (int i = 0; i < det_result.objNum; i++) {
    RockIvaObjectInfo* obj = &det_result.objInfo[i];
    printf("%d cls=%d score=%d box=(%d %d %d %d)\n", i, obj->type, obj->score,
        obj->rect.topLeft.x, obj->rect.topLeft.y, obj->rect.bottomRight.x, obj->rect.bottomRight.y);

    int left = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.topLeft.x);
    int top = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.topLeft.y);
    int right = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.bottomRight.x);
    int bottom = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.bottomRight.y);

    RockXDrawRectangle(image.buffer.virtAddr, image.width, image.height, image.format,
        left, top, right - left + 1, bottom - top + 1, COLOR_GREEN, 2);
}

write_image_jpeg("output.jpg", 100, &image);

free(image.buffer.virtAddr);

RockXIvaReleaseTaskDet(&handle);

return 0;
}

```

3.2.1.2 IvaTracker

【功能】

目标跟踪

【模块名】

IvaTracker

【参数】

参数名称	类型	说明
task_params	RockIvaDetTaskParams	

【输入】

输入名称	类型	说明
detect_result	RockIvaDetectResult	

【输出】

输出名称	类型	说明
filterd_result	RockIvaDetectResult	过滤结果
tracked_result	RockIvaDetectResult	跟踪结果

【示例】

```
int RockXIvaInitTaskTracker(RockXHandle* rockxhandle, const RockIvaDetTaskParams* params)
{
    RockXRetCode ret;

    RockXHandle handle = RockXCreate();

    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "IvaTracker", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_DATA_PATH, "/data/rockiva_data", 0);

    // set rknn model name
    ret = RockXSetParamPointer(handle, "IvaTracker", "task_params", params, 1, NULL);

    ret = RockXInit(handle);
    if (ret != ROCKX_RET_SUCCESS) {
        printf("RockXInit fail, ret=%d\n", ret);
        return ret;
    }

    *rockxhandle = handle;

    return 0;
}

int RockXIvaReleaseTaskTracker(RockXHandle* rockxhandle)
{
    RockXDestroy(*rockxhandle);
    *rockxhandle = NULL;
    return 0;
}

int main(int argc, char* argv[]) {
    char* det_model_name = argv[1];
    char* image_path = argv[2];

    int ret;

    // create handle
```

```

RockXHandle det_handle;
RockXHandle track_handle;

ret = RockXIvaInitTaskDet(&det_handle, det_model_name);
if (ret < 0) {
    printf("RockXIvaInitTaskDet fail, ret=%d\n", ret);
    return ret;
}

RockIvaDetTaskParams* detParams = malloc(sizeof(RockIvaDetTaskParams));
memset(detParams, 0, sizeof(RockIvaDetTaskParams));
detParams->detObjectType |= ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);
ret = RockXIvaInitTaskTracker(&track_handle, detParams);
if (ret < 0) {
    printf("RockXIvaInitTaskTracker fail, ret=%d\n", ret);
    return ret;
}

// read image
RockXImage image;
memset(&image, 0, sizeof(RockXImage));
read_image_jpeg(image_path, &image);

RockIvaDetectResult det_result;
ret = RockXIvaCallTaskDet(det_handle, &image, &det_result);
if (ret < 0) {
    printf("RockXIvaCallTaskDet fail, ret=%d\n", ret);
    return ret;
}

RockIvaDetectResult track_result;
ret = RockXIvaCallTaskTracker(track_handle, &det_result, &track_result);
if (ret < 0) {
    printf("RockXIvaCallTaskTracker fail, ret=%d\n", ret);
    return ret;
}

for (int i = 0; i < track_result.objNum; i++) {
    RockIvaObjectInfo* obj = &track_result.objInfo[i];
    printf("%d id=%d cls=%d score=%d box=(%d %d %d %d)\n", i, obj->objId, obj->type,
obj->score,
        obj->rect.topLeft.x, obj->rect.topLeft.y, obj->rect.bottomRight.x, obj-
>rect.bottomRight.y);

    int left = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.topLeft.x);
    int top = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.topLeft.y);
    int right = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.bottomRight.x);
    int bottom = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.bottomRight.y);

    RockXDrawRectangle(image.buffer.virtAddr, image.width, image.height, image.format,
        left, top, right - left + 1, bottom - top + 1, COLOR_GREEN, 2);
}

```

```

    write_image_jpeg("output.jpg", 100, &image);

    free(image.buffer.virtAddr);

    RockXIvaReleaseTaskDet(&det_handle);
    RockXIvaReleaseTaskTracker(&track_handle);

    return 0;
}

```

3.2.1.3 IvaPeridet

【功能】

周界检测，支持区域入侵、区域进入、区域离开、越界四种类型

【模块名】

IvaPeridet

【参数】

参数名称	类型	说明
task_params	RockIvaBaTaskParams	周界参数配置

【输入】

参数名称	类型	说明
track_objects	RockIvaDetectResult	目标跟踪结果

【输出】

参数输出	类型	说明
peridet_result	RockIvaBaResult	周界检测结果

【示例】

```

int RockXIvaInitTaskBa(RockXHandle* rockxhandle, const RockIvaBaTaskParams* params)
{
    RockXRetCode ret;

    RockXHandle handle = RockXCreate();

    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "IvaPeriDet", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_DATA_PATH, "/data/rockiva_data", 0);
}

```

```

// set rknn model name
ret = RockXSetParamPointer(handle, "IvaPeriDet", "task_params", params, 1, NULL);

ret = RockXInit(handle);
if (ret != ROCKX_RET_SUCCESS) {
    printf("RockXInit fail, ret=%d\n", ret);
    return ret;
}

*rockxhandle = handle;
return 0;
}

int RockXIvaReleaseTaskBa(RockXHandle* rockxhandle)
{
    RockXDestroy(*rockxhandle);
    *rockxhandle = NULL;
    return 0;
}

int main(int argc, char* argv[]) {
    char* det_model_name = argv[1];
    char* image_path = argv[2];

    int ret;

    // create handle
    RockXHandle det_handle;
    RockXHandle track_handle;
    RockXHandle ba_handle;

    ret = RockXIvaInitTaskDet(&det_handle, det_model_name);
    if (ret < 0) {
        printf("RockXIvaInitTaskDet fail, ret=%d\n", ret);
        return ret;
    }

    RockIvaDetTaskParams* detParams = malloc(sizeof(RockIvaDetTaskParams));
    memset(detParams, 0, sizeof(RockIvaDetTaskParams));
    detParams->detObjectType |= ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);
    ret = RockXIvaInitTaskTracker(&track_handle, detParams);
    if (ret < 0) {
        printf("RockXIvaInitTaskTracker fail, ret=%d\n", ret);
        return ret;
    }

    RockIvaBaTaskParams* baParams = malloc(sizeof(RockIvaBaTaskParams));
    memset(baParams, 0, sizeof(RockIvaBaTaskParams));
    baParams->aiConfig.detectResultMode = 1;
    baParams->aiConfig.filterPersonMode = 1;

    // 构建一个区域入侵规则
    baParams->baRules.areaInBreakRule[0].ruleEnable = 1;

```

```

    baParams->baRules.areaInBreakRule[0].alertTime = 1000; // 1000ms
    baParams->baRules.areaInBreakRule[0].ruleID = 1;
    baParams->baRules.areaInBreakRule[0].objType |=
ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);
    baParams->baRules.areaInBreakRule[0].area.pointNum = 4;
    baParams->baRules.areaInBreakRule[0].area.points[0].x =
ROCKIVA_PIXEL_RATION_CONVERT(1920, 0);
    baParams->baRules.areaInBreakRule[0].area.points[0].y =
ROCKIVA_PIXEL_RATION_CONVERT(1080, 0);
    baParams->baRules.areaInBreakRule[0].area.points[1].x =
ROCKIVA_PIXEL_RATION_CONVERT(1920, 1920);
    baParams->baRules.areaInBreakRule[0].area.points[1].y =
ROCKIVA_PIXEL_RATION_CONVERT(1080, 0);
    baParams->baRules.areaInBreakRule[0].area.points[2].x =
ROCKIVA_PIXEL_RATION_CONVERT(1920, 1920);
    baParams->baRules.areaInBreakRule[0].area.points[2].y =
ROCKIVA_PIXEL_RATION_CONVERT(1080, 1080);
    baParams->baRules.areaInBreakRule[0].area.points[3].x =
ROCKIVA_PIXEL_RATION_CONVERT(1920, 0);
    baParams->baRules.areaInBreakRule[0].area.points[3].y =
ROCKIVA_PIXEL_RATION_CONVERT(1080, 1080);
    ret = RockXIvaInitTaskBa(&ba_handle, baParams);
    if (ret < 0) {
        printf("RockXIvaInitTaskBa fail, ret=%d\n", ret);
        return ret;
    }

    // read image
    RockXImage image;
    memset(&image, 0, sizeof(RockXImage));
    read_image_jpeg(image_path, &image);

    RockIvaDetectResult det_result;
    ret = RockXIvaCallTaskDet(det_handle, &image, &det_result);
    if (ret < 0) {
        printf("RockXIvaCallTaskDet fail, ret=%d\n", ret);
        return ret;
    }

    RockIvaDetectResult track_result;
    ret = RockXIvaCallTaskTracker(track_handle, &det_result, &track_result);
    if (ret < 0) {
        printf("RockXIvaCallTaskTracker fail, ret=%d\n", ret);
        return ret;
    }

    RockIvaBaResult ba_result;
    ret = RockXIvaCallTaskBa(ba_handle, &image, &track_result, &ba_result);
    if (ret < 0) {
        printf("RockXIvaCallTaskBa fail, ret=%d\n", ret);
        return ret;
    }
}

```



```

    for (int i = 0; i < ba_result.objNum; i++) {
        RockIvaBaObjectInfo* ba_obj = &ba_result.triggerObjects[i];
        RockIvaObjectInfo* obj = &ba_obj->objInfo;
        printf("%d id=%d cls=%d score=%d box=(%d %d %d %d)\n", i, obj->objId, obj->type,
obj->score,
                obj->rect.topLeft.x, obj->rect.topLeft.y, obj->rect.bottomRight.x, obj-
>rect.bottomRight.y);

        int left = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.topLeft.x);
        int top = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.topLeft.y);
        int right = ROCKIVA_RATIO_PIXEL_CONVERT(image.width, obj->rect.bottomRight.x);
        int bottom = ROCKIVA_RATIO_PIXEL_CONVERT(image.height, obj->rect.bottomRight.y);

        RockXDrawRectangle(image.buffer.virtAddr, image.width, image.height, image.format,
                left, top, right - left + 1, bottom - top + 1, COLOR_GREEN, 2);
    }

    write_image_jpeg("output.jpg", 100, &image);

    free(image.buffer.virtAddr);

    RockXIvaReleaseTaskDet(&det_handle);
    RockXIvaReleaseTaskTracker(&track_handle);
    RockXIvaReleaseTaskBa(&ba_handle);

    return 0;
}

```

